

Proyecto TechDAM - **JDBC Avanzado**

FECHA: 16/11/2025

GitHub: https://github.com/marcospalomomendez/palomomendez_marcos_techdam.git

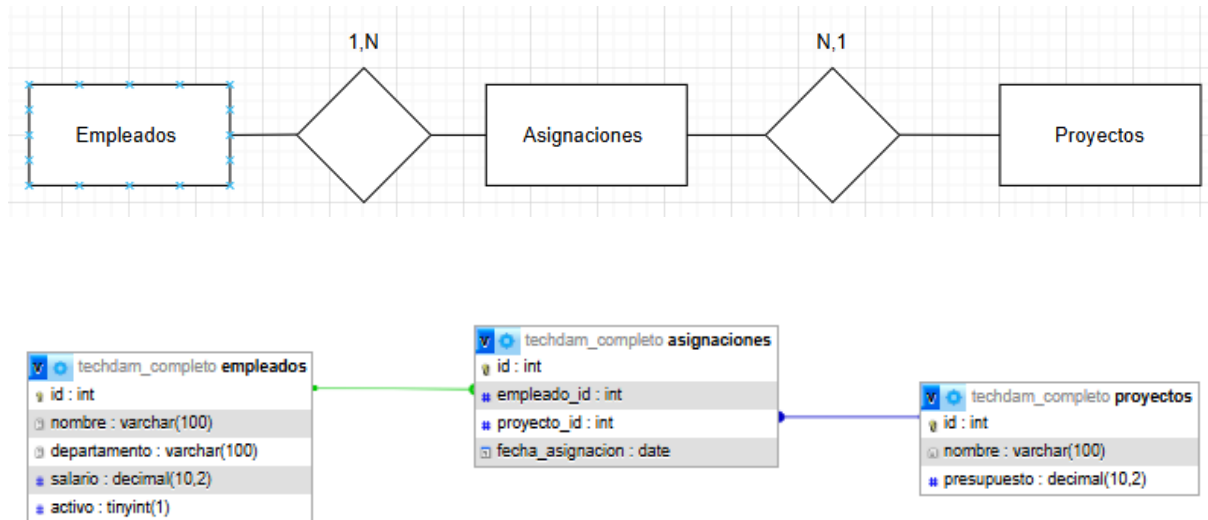
MARCOS PALOMO MÉNDEZ

INDICE

DIAGRAMA DE ER.....	3
PRIMARY KEY Y FOREIGN KEY.....	3
Tabla empleados.....	3
Tabla proyectos.....	3
Tabla asignaciones.....	3
TABLAS CREADAS CON MYSQL.....	4
Tabla empleados.....	4
Tabla proyectos.....	4
Tabla asignaciones.....	4
EJECUCIÓN EXITOSA DE CRUD DE EMPLEADOS.....	5
CONSOLA.....	5
OBTENER TODOS LOS EMPLEADOS.....	5
CREAR UN EMPLEADO NUEVO.....	6
ACTUALIZAR UN EMPLEADO POR ID.....	6
ELIMINAR UN EMPLEADO.....	6
OBTENER LOS EMPLEADOS DESPUÉS DE MODIFICACIONES.....	6
CÓDIGO.....	7
EJECUCIÓN EXITOSA DE CRUD DE PROYECTOS.....	8
CONSOLA.....	8
OBTENER TODOS LOS PROYECTOS.....	8
CREAR UN PROYECTO NUEVO.....	9
ACTUALIZAR UN PROYECTO POR ID.....	9
OBTENER UN PROYECTO POR ID.....	9
ELIMINAR UN PROYECTO.....	9
OBTENER LOS PROYECTO DESPUÉS DE MODIFICACIONES.....	9
CÓDIGO.....	10
INVOCACIÓN PROCEDIMIENTOS ALMACENADOS.....	11
CONSOLA.....	11
PROCEDIMIENTO actualizar_salario_departamento.....	11
PROCEDIMIENTO contar_asignaciones.....	11
CÓDIGO.....	12
INVOCACIÓN TRANSACCIONES.....	12
CONSOLA.....	12
CÓDIGO.....	12
¿Por qué usar PreparedStatement en lugar de Statement?.....	13
¿Qué ventajas tiene el pool de conexiones?.....	13
¿En qué caso usarías procedimientos almacenados?.....	13
¿Por qué es importante el control de transacciones?.....	13
Realizar una transacción.....	14
Problema con conexión con la base de datos.....	14

BASE DE DATOS

DIAGRAMA DE ER



PRIMARY KEY Y FOREIGN KEY

Tabla empleados

Primary key → id

Tabla proyectos

Primary key → id

Tabla asignaciones

Primary key → id

Foreign key → empleado_id hace referencia a empleados(id)

Foreign key → proyecto_id hace referencia a proyectos(id)

CAPTURA DE PANTALLA

TABLAS CREADAS CON MYSQL

Tabla empleados

`SELECT * FROM `empleados``

☐ Perfilando [\[Editar en línea \]](#) [\[Editar \]](#) [\[Explicar SQL \]](#) [\[Crear código PHP \]](#) [\[Actualizar \]](#)

☐ Mostrar todo | Número de filas: 25 | Filtrar filas:

Opciones extra

				id	nombre	departamento	salario	activo
☐				1	Ana Torres	IT	2200.00	1
☐				2	Luis Pérez	Marketing	1800.00	1
☐				3	Marta Gómez	IT	2400.00	1
☐				4	Carlos Ruiz	Ventas	1600.00	1
☐				5	Laura Díaz	IT	2500.00	1

Tabla proyectos

`SELECT * FROM `proyectos``

☐ Perfilando [\[Editar en línea \]](#) [\[Editar \]](#) [\[Explicar SQL \]](#) [\[Crear código PHP \]](#) [\[Actualizar \]](#)

☐ Mostrar todo | Número de filas: 25 | Filtrar filas:

Opciones extra

				id	nombre	presupuesto
☐				1	Migración Servidores	20000.00
☐				2	Campaña Q1	10000.00
☐				3	App Móvil	50000.00
☐				4	Web Corporativa	15000.00
☐				5	Auditoría Interna	8000.00

Tabla asignaciones

`SELECT * FROM `asignaciones``

☐ Perfilando [\[Editar en línea \]](#) [\[Editar \]](#) [\[Explicar SQL \]](#) [\[Crear código PHP \]](#) [\[Actualizar \]](#)

☐ Mostrar todo | Número de filas: 25 | Filtrar filas: Ordenar

Opciones extra

				id	empleado_id	proyecto_id	fecha_asignacion
☐				1	1	1	2025-11-16
☐				2	2	2	2025-11-16
☐				3	3	3	2025-11-16
☐				4	4	4	2025-11-16
☐				5	3	3	2025-11-16

EJECUCIÓN EXITOSA DE CRUD DE EMPLEADOS

CONSOLA

```
--- CRUD EMPLEADOS ---
LISTA INICIAL DE EMPLEADOS:
SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinder for further details.
Pool de conexiones HikariCP configurado correctamente
Empleado{id=1, nombre='Ana Torres', departamento='IT', salario=2200.00, activo=true}
Empleado{id=2, nombre='Luis Pérez', departamento='Marketing', salario=1800.00, activo=true}
Empleado{id=3, nombre='Marta Gómez', departamento='IT', salario=2400.00, activo=true}
Empleado{id=4, nombre='Carlos Ruiz', departamento='Ventas', salario=1600.00, activo=true}
Empleado{id=5, nombre='Laura Díaz', departamento='IT', salario=2500.00, activo=true}
Empleado creado con ID: 6
Actualizado: true
Optional[Empleado{id=6, nombre='Ana Torres', departamento='IT', salario=2600.00, activo=true}]
Empleado{id=1, nombre='Ana Torres', departamento='IT', salario=2200.00, activo=true}
Empleado{id=2, nombre='Luis Pérez', departamento='Marketing', salario=1800.00, activo=true}
Empleado{id=3, nombre='Marta Gómez', departamento='IT', salario=2400.00, activo=true}
Empleado{id=4, nombre='Carlos Ruiz', departamento='Ventas', salario=1600.00, activo=true}
Empleado{id=5, nombre='Laura Díaz', departamento='IT', salario=2500.00, activo=true}
Empleado{id=6, nombre='Ana Torres', departamento='IT', salario=2600.00, activo=true}
Eliminando empleado con ID: 5
LISTA FINAL DE EMPLEADOS:
Empleado{id=1, nombre='Ana Torres', departamento='IT', salario=2200.00, activo=true}
Empleado{id=2, nombre='Luis Pérez', departamento='Marketing', salario=1800.00, activo=true}
Empleado{id=3, nombre='Marta Gómez', departamento='IT', salario=2400.00, activo=true}
Empleado{id=4, nombre='Carlos Ruiz', departamento='Ventas', salario=1600.00, activo=true}
Empleado{id=6, nombre='Ana Torres', departamento='IT', salario=2600.00, activo=true}
```

OBTENER TODOS LOS EMPLEADOS

Llamo al método obtenerTodos y como devuelve una lista posteriormente lo muestro aprovechando el toList de Empleado.

```
Empleado{id=1, nombre='Ana Torres', departamento='IT', salario=2200.00, activo=true}
Empleado{id=2, nombre='Luis Pérez', departamento='Marketing', salario=1800.00, activo=true}
Empleado{id=3, nombre='Marta Gómez', departamento='IT', salario=2400.00, activo=true}
Empleado{id=4, nombre='Carlos Ruiz', departamento='Ventas', salario=1600.00, activo=true}
Empleado{id=5, nombre='Laura Díaz', departamento='IT', salario=2500.00, activo=true}
```

CREAR UN EMPLEADO NUEVO

Llamo al método crear y como devuelve un int que es el id del que se ha creado lo muestro.

```
Empleado creado con ID: 6
```

ACTUALIZAR UN EMPLEADO POR ID

Llamo al método eliminar y como devuelve un booleano true si lo ha eliminado y false si no lo muestro.

```
Actualizado: true  
Optional[Empleado{id=6, nombre='Ana Torres', departamento='IT', salario=2600.00, activo=true}]
```

ELIMINAR UN EMPLEADO

Llamo al método eliminar y como devuelve un int que es el id del que se ha creado lo muestro.

```
Eliminando empleado con ID: 5
```

OBTENER LOS EMPLEADOS DESPUÉS DE MODIFICACIONES

Llamo al método obtenerTodos y como devuelve una lista posteriormente lo muestro aprovechando el toList de Proyecto.

```
Eliminando empleado con ID: 5  
LISTA FINAL DE EMPLEADOS:  
Empleado{id=1, nombre='Ana Torres', departamento='IT', salario=2200.00, activo=true}  
Empleado{id=2, nombre='Luis Pérez', departamento='Marketing', salario=1800.00, activo=true}  
Empleado{id=3, nombre='Marta Gómez', departamento='IT', salario=2400.00, activo=true}  
Empleado{id=4, nombre='Carlos Ruiz', departamento='Ventas', salario=1600.00, activo=true}  
Empleado{id=6, nombre='Ana Torres', departamento='IT', salario=2600.00, activo=true}
```

CÓDIGO

```
public static void testEmpleados() {
    System.out.println("\n--- CRUD EMPLEADOS ---");

    EmpleadoDAO dao = new EmpleadoDAO();
    System.out.println("LISTA INICIAL DE EMPLEADOS:");
    // Listar
    dao.obtenerTodos().forEach(System.out::println);
    // Crear
    Empleado emp = new Empleado();
    emp.setNombre("Ana Torres");
    emp.setDepartamento("IT");
    emp.setSalario(new BigDecimal("2200"));
    emp.setActivo(true);
    int id = dao.crear(emp);
    System.out.println("Empleado creado con ID: " + id);

    // Actualizar
    emp.setId(id);
    emp.setSalario(new BigDecimal("2600"));
    System.out.println("Actualizado: " + dao.actualizar(emp));

    // Mostrar por ID
    System.out.println(dao.obtenerPorId(id));
    dao.obtenerTodos().forEach(System.out::println);

    // Eliminar un empleado
    int id_eliminar = 5;
    System.out.println("Eliminando empleado con ID: " +
id_eliminar);
    boolean eliminado = dao.eliminar(id_eliminar);

    System.out.println("LISTA FINAL DE EMPLEADOS:");
    // Listar
    dao.obtenerTodos().forEach(System.out::println);
}
```

EJECUCIÓN EXITOSA DE CRUD DE PROYECTOS

CONSOLA

```
--- CRUD PROYECTOS ---  
LISTA INICIAL DE PROYECTO:  
Proyecto{id=1, nombre='Migración Servidores', salario=20000.00}  
Proyecto{id=2, nombre='Campaña Q1', salario=10000.00}  
Proyecto{id=3, nombre='App Móvil', salario=50000.00}  
Proyecto{id=4, nombre='Web Corporativa', salario=15000.00}  
Proyecto{id=5, nombre='Auditoría Interna', salario=8000.00}  
Proyecto creado con ID: 6  
Actualizado: true  
Optional[Proyecto{id=6, nombre='Autoamatzacion de Procesos', salario=250000.00}]  
Proyecto{id=1, nombre='Migración Servidores', salario=20000.00}  
Proyecto{id=2, nombre='Campaña Q1', salario=10000.00}  
Proyecto{id=3, nombre='App Móvil', salario=50000.00}  
Proyecto{id=4, nombre='Web Corporativa', salario=15000.00}  
Proyecto{id=5, nombre='Auditoría Interna', salario=8000.00}  
Proyecto{id=6, nombre='Autoamatzacion de Procesos', salario=250000.00}  
Eliminando proyecto con ID: 5  
LISTA FINAL DE PROYECTOS:  
Proyecto{id=1, nombre='Migración Servidores', salario=20000.00}  
Proyecto{id=2, nombre='Campaña Q1', salario=10000.00}  
Proyecto{id=3, nombre='App Móvil', salario=50000.00}  
Proyecto{id=4, nombre='Web Corporativa', salario=15000.00}  
Proyecto{id=6, nombre='Autoamatzacion de Procesos', salario=250000.00}
```

OBTENER TODOS LOS PROYECTOS

Llamo al método obtenerTodos y como devuelve una lista posteriormente lo muestro aprovechando el toList de Proyecto.

```
Proyecto{id=1, nombre='Migración Servidores', salario=20000.00}  
Proyecto{id=2, nombre='Campaña Q1', salario=10000.00}  
Proyecto{id=3, nombre='App Móvil', salario=50000.00}  
Proyecto{id=4, nombre='Web Corporativa', salario=15000.00}  
Proyecto{id=5, nombre='Auditoría Interna', salario=8000.00}
```

CREAR UN PROYECTO NUEVO

Llamo al método crear y como devuelve un int que es el id del que se ha creado lo muestro.

```
Proyecto creado con ID: 6
```

ACTUALIZAR UN PROYECTO POR ID

Llamo al método eliminar y como devuelve un booleano true si lo ha eliminado y false si no lo muestro.

```
Actualizado: true
```

OBTENER UN PROYECTO POR ID

Llamo al método obtenerPorID y como devuelve un Optional en este caso de tipo Proyecto lo muestro.

```
Optional[Proyecto{id=6, nombre='Autoamatizacion de Procesos', salario=250000.00}]
```

ELIMINAR UN PROYECTO

Llamo al método eliminar y como devuelve un int que es el id del que se ha creado lo muestro.

```
Eliminando proyecto con ID: 5
```

OBTENER LOS PROYECTO DESPUÉS DE MODIFICACIONES

Llamo al método obtenerTodos y como devuelve una lista posteriormente lo muestro aprovechando el toList de Proyecto.

LISTA FINAL DE PROYECTOS:

```
Proyecto{id=1, nombre='Migración Servidores', salario=20000.00}  
Proyecto{id=2, nombre='Campaña Q1', salario=10000.00}  
Proyecto{id=3, nombre='App Móvil', salario=50000.00}  
Proyecto{id=4, nombre='Web Corporativa', salario=15000.00}  
Proyecto{id=6, nombre='Autoamatizacion de Procesos', salario=250000.00}
```

CÓDIGO

```
public static void testProyectos() {  
    System.out.println("\n--- CRUD PROYECTOS ---");  
  
    ProyectoDAO dao = new ProyectoDAO();  
    System.out.println("LISTA INICIAL DE PROYECTO:");  
    // Listar  
    dao.obtenerTodos().forEach(System.out::println);  
    // Crear  
    Proyecto proy = new Proyecto();  
    proy.setNombre("Autoamatizacion de Procesos");  
    proy.setPresupuesto(new BigDecimal("20200"));  
    int id = dao.crear(proy);  
    System.out.println("Proyecto creado con ID: " + id);  
  
    // Actualizar  
    proy.setId(id);  
    proy.setPresupuesto(new BigDecimal("250000"));  
    System.out.println("Actualizado: " + dao.actualizar(proy));  
  
    // Mostrar por ID  
    System.out.println(dao.obtenerPorId(id));  
    dao.obtenerTodos().forEach(System.out::println);  
  
    // Eliminar un proyecto  
    int id_eliminar = 5;  
    System.out.println("Eliminando proyecto con ID: " + id_eliminar);  
    boolean eliminado = dao.eliminar(id_eliminar);  
  
    System.out.println("LISTA FINAL DE PROYECTOS:");  
    // Listar  
    dao.obtenerTodos().forEach(System.out::println);  
}
```

INVOCACIÓN PROCEDIMIENTOS ALMACENADOS

CONSOLA

```
--- CRUD PROCEDIMIENTOS ---  
Procedimiento ejecutado: 3 empleados actualizados  
LISTA DE EMPLEADOS DESPUES DE AUMENTO DE SALARIO:  
Empleado{id=1, nombre='Ana Torres', departamento='IT', salario=2761.00, activo=true}  
Empleado{id=2, nombre='Luis Pérez', departamento='Marketing', salario=1800.00, activo=true}  
Empleado{id=3, nombre='Marta Gómez', departamento='IT', salario=3012.00, activo=true}  
Empleado{id=4, nombre='Carlos Ruiz', departamento='Ventas', salario=1600.00, activo=true}  
Empleado{id=6, nombre='Ana Torres', departamento='IT', salario=3263.00, activo=true}  
Procedimiento ejecutado: Empleado 3 tiene 2 asignaciones
```

PROCEDIMIENTO actualizar_salario_departamento

Llamo al procedimiento y muestro como quedan los empleados después de hacer el procedimiento

```
Procedimiento ejecutado: 3 empleados actualizados  
LISTA DE EMPLEADOS DESPUES DE AUMENTO DE SALARIO:  
Empleado{id=1, nombre='Ana Torres', departamento='IT', salario=2761.00, activo=true}  
Empleado{id=2, nombre='Luis Pérez', departamento='Marketing', salario=1800.00, activo=true}  
Empleado{id=3, nombre='Marta Gómez', departamento='IT', salario=3012.00, activo=true}  
Empleado{id=4, nombre='Carlos Ruiz', departamento='Ventas', salario=1600.00, activo=true}  
Empleado{id=6, nombre='Ana Torres', departamento='IT', salario=3263.00, activo=true}
```

PROCEDIMIENTO contar_asignaciones

Llamo al procedimiento y muestro cuántas asignaciones tiene el empleado con id = 3

```
Procedimiento ejecutado: Empleado 3 tiene 2 asignaciones
```

CÓDIGO

```
public static void testProcedimientos() {
    System.out.println("\n--- CRUD PROCEDIMIENTOS ---");
    ProcedimientosService ps = new ProcedimientosService();
    EmpleadoDAO dao = new EmpleadoDAO();
    ps.actualizarSalariosDepartamento("IT", 25.5 );
    System.out.println("LISTA DE EMPLEADOS DESPUES DE AUMENTO DE
SALARIO:");
    dao.obtenerTodos().forEach(System.out::println);
    ps.contar_asignaciones(3);
}
```

INVOCACIÓN TRANSACCIONES

CONSOLA

```
--- CRUD TRANSACCIONES ---
LISTA DE PROYECTOS DESPUES DE LA TRANSACCIÓN:
Proyecto{id=1, nombre='Migración Servidores', salario=15000.00}
Proyecto{id=2, nombre='Campaña Q1', salario=15000.00}
Proyecto{id=3, nombre='App Móvil', salario=50000.00}
Proyecto{id=4, nombre='Web Corporativa', salario=15000.00}
Proyecto{id=6, nombre='Autoamatizacion de Procesos', salario=250000.00}
Error asignando empleado 5, rollback parcial.
Error asignando empleado 999, rollback parcial.
```

CÓDIGO

```
public static void testTransacciones() {
    System.out.println("\n--- CRUD TRANSACCIONES ---");
    TransaccionesService ts = new TransaccionesService();
    ProyectoDAO dao = new ProyectoDAO();
    ts.transferirPresupuesto(1, 2, new BigDecimal("5000"));
    System.out.println("LISTA DE PROYECTOS DESPUES DE LA TRANSACCIÓN:");
}
```

```
dao.obtenerTodos().forEach(System.out::println);  
List<Integer> empleadoIds = List.of(3,4,5,999,6);  
ts.asignarEmpleadosConSavepoint(1,empleadoIds);  
}
```

EXPLICACIÓN DE DECISIONES

TÉCNICAS

¿Por qué usar PreparedStatement en lugar de Statement?

- **Seguridad:** Previene ataques de **SQL Injection**, ya que los parámetros se envían de forma segura y no concatenados en la consulta.
- **Legibilidad y mantenimiento:** Separar la consulta de los parámetros hace que el código sea más claro y fácil de mantener.
- **Flexibilidad:** Permite trabajar con distintos tipos de datos (strings, enteros, fechas) sin preocuparse por el formato exacto en la concatenación.

¿Qué ventajas tiene el pool de conexiones?

- **Rendimiento:** Evita el coste de crear y destruir conexiones cada vez que se accede a la base de datos.
- **Escalabilidad:** Permite que múltiples usuarios compartan un conjunto limitado de conexiones, optimizando recursos.
- **Reutilización:** Una vez que una conexión se libera, puede ser usada por otro proceso sin necesidad de abrir una nueva.

¿En qué caso usarías procedimientos almacenados?

- **Reutilización:** Se pueden invocar desde distintas aplicaciones sin duplicar código.
- **Mantenimiento:** Si cambias la lógica, basta con modificar el procedimiento sin tocar el código de las aplicaciones.

¿Por qué es importante el control de transacciones?

- **Consistencia:** Garantiza que los datos permanezcan coherentes incluso si ocurre un error.
- **Atomicidad:** Una transacción se ejecuta completa o no se ejecuta en absoluto (todo o nada).
- **Durabilidad:** Una vez confirmada (commit), la transacción queda registrada de forma permanente, incluso ante fallos del sistema.

PROBLEMAS ENCONTRADOS Y SOLUCIONES

Realizar una transacción

Al realizar la transacción de `asignarEmpleadoConSavepoint` he tenido problemas al hacer el rollback ya que en `Asignar.DAO` no lanzaba la excepción la capturaba. La solución ha sido en vez de capturar esta la he lanzado y la he capturado posteriormente en el método `asignarEmpleadoConSavepoint`.

Problema con conexión con la base de datos

En el momento en el que estaba elaborando el trabajo he tenido un problema con la conexión con la base de datos ya que se me había olvidado actualizar las dependencias del `pom.xml`.